

Natural language processing

Information retrieval

Finn Årup Nielsen

DTU Compute, Technical University of Denmark

March 6, 2026

Information retrieval

Given a query find documents/passages stored in a database.

Sparse: classical information retrieval with keywords-based search.

Dense: Newer information retrieval where text is converted to a dense vectorial representation with a large language model.

Hybrid: Combination of sparse and dense retrieval.

Example corpus

A small corpus: Five documents, here just each an English sentence.

```
1 corpus = [  
2     "Natural language processing enables computers to understand human language",  
3     "Transformers have revolutionized the field of deep learning in NLP",  
4     "Tokenization is a fundamental step in text preprocessing",  
5     "DTU researchers are exploring multilingual models for information retrieval",  
6     "Pretrained language models can be fine-tuned for various NLP tasks"  
7 ]
```

Terms: a, are, be, can, deep, enables, field, for, have, human, language, natural, processing, ...

Sparse information retrieval

Document-term matrix: (documents-times-terms)-sized matrix with counts of the number of times a particular term (word/token/character) appears in a document: bag-of-words matrix — often sparse.

	computers	deep	dtu	enables	exploring	field	fine-tuned	fundamental	human	information	language	learning	models	multilingual	natural	nlp	preprocessing	pretrained	processing	researchers	retrieval	revolutionized	step	tasks	text	tokenization	transformers	understand	various
D1	1			1					1	2					1			1										1	
D2		1				1						1				1						1					1		
D3							1										1						1		1	1			
D4			1		1				1				1	1						1	1								
D5						1				1	1	1	1		1		1	1						1				1	

How to represent sparse data?

Python Numpy array (probably not that good an idea for sparse data)

Python Scipy sparse array

Python libraries: scikit-learn, Gensim, bm25s

Dedicated search engines, e.g., Elastic search.

What is a word/token

One or more word characters with regular expression: `\w+`. Default of scikit-learn's `CountVectorizer` is `\b\w\w+\b` (two or more word characters).

Conversion of upper/lowercase to common case. lowercase is default in scikit-learn's `CountVectorizer`.

Accent stripping, e.g., `á` $\rightarrow$ `a`.

Lemmatization or stemming: `start`, `starting`, `started`, `starts` $\rightarrow$ `start`. Lemmatization requires a dictionary of inflections.

Stop words removal. Stop words are common words (`the`, `a`, `for`, ...). scikit-learn's `CountVectorizer` has a non-default English stop word list.

Max number of terms, or discarding rare terms.

N-gram

Count the occurrence of consecutive pairs/triples/. . . of words or characters.

Example: “Natural language processing enables computers to understand human language”

Terms for bigram (2-gram): natural, natural language, language, language processing, processing, processing enables, . . . ,

Terms for trigram (3-gram): natural, natural language, natural language processing, language, . . .

Good to represent multiword expressions such as *magnetic resonance imaging*, *natural language processing*

`ngram_range` parameter in scikit-learn's `CountVectorizer`, Phrases in `GenSim`.

Normalization of matrix

Term-frequency-inverse document frequency (TF-IDF)

Term-frequency (TF): The number of counts perhaps scaled with the logarithm.

Inverse document frequency (IDF): $\log(N/df)$ or similar

Normalize the length of the vectors, e.g., with the L2-norm for cosine similarity computation.

TF scaling: A document that has N occurrences of a term are not N times as relevant for the query as a document that has N occurrences.

IDF scaling: Downscale words that occur across the corpus. Example 1000 documents. 'and' $\log(1000/1000) = 0$, uncommon word that occurs in 0.2 % of the documents: $\log(1000/2) \approx 6$.

Query modification

Spelling corrections, e.g., by edit distance.

Query expansion: Add, e.g., synonyms to the query

Dense information retrieval

Convert the text to a vectorial “semantic” representation by “embedding” it with a language model.

The text can be a full document or passages (chunks). Often chunks because the window of the embedding model is limited.

Embeddings are usually the values of one of the last layers of a large language models.

There are pre-trained models available, e.g., via `SentenceTransformer` or through LLM APIs.

Features of embedding models: Languages supported, number of parameters, size of (context) window (how many tokens can be embedded), dimension of embedding space (usually some hundreds), whether the embedding vector is normalized (already), task prefix (e.g., “`search_query`”).

CampusAI embedding

Currently nomic-embed-text is installed in CampusAI. Example

```
1 texts = [  
2     "Natural language processing is a field of artificial intelligence.",  
3     "Transformers are widely used for language modeling.",  
4     "Prime Minister Mette Frederiksen flew to Nuuk.",  
5     "Jens-Frederik Nielsen is both a badminton player and a politician.",  
6 ]  
7  
8 # Create embeddings  
9 response = client.embeddings.create(  
10     model="nomic-embed-text",  
11     input=texts  
12 )  
13  
14 # Compute similarities  
15 import numpy as np  
16 embeddings = np.array([embed.embedding for embed in response.data])  
17 similarities = embeddings @ embeddings.T
```

Dense storage

Numpy array, do not use sparse.

Dedicated vector storage.

...

Match query with documents

Query q and a document represented in a vector x and the cosine similarity

$$\text{sim}_{\text{cosine}}(x, q) = \frac{x'q}{\|x\|_2 \|q\|_2},$$

where $\|x\|_2$ is the Euclidean norm, also called the $L2$ -norm. You often normalized the vectors in advance: There is no reason to normalize x at each query, — instead do it once and for all. And some tools, LLM API and scikit-learn, may return the vectors already normalized. The for an entire corpus in a matrix, X , is then just a matrix-vector product

$$Xq.$$

Hybrid information retrieval

Hybrid = Sparse retrieval + dense retrieval

Make a combined ranked list with weight averaging on the cosine similarities.

$$s_{d,\text{hybrid}} = \alpha s_{d,\text{dense}} + (1 - \alpha) s_{d,\text{sparse}}, \quad (1)$$

or reciprocal rank fusion

$$s_{d,\text{RFF}} = \sum_r \frac{1}{k + \text{rank}_{d,r}}, \quad (2)$$

Data for exercise

I have crawled the 2025 DTU Course database and data are available in two forms:

WARC HTML archival format.

JSON lines file where some of the information from the HTML has been extracted.

There might be some course information missing, e.g., courses shared with the University of Copenhagen and 01001 is seeming also missing.

You must not put the DTU Course information on a public website, e.g., a public GitHub repository.

Where does it lead to?

How can this be combined with LLMs?

Retrieval-augmented generation: LLM + information retrieval (next session)

Handling of long-tail information: The information retrieval system has access to the specialized document.

Quicker update: When new information arrive it can just be added to the document store. If the LLM were to handle this it would require an expensive retraining.

What is missing?

Tabular data

Images

Latent semantic indexing